

Physics Simulation: High Resolution Hot Jupiter Atmospheric Retrieval

Goal: In this project I will be analysing the detection of molecules of CO and H₂O in a hot Jupiter's atmosphere.

Plan: As an introduction to the topic the plan will be to create a model spectra of light on a plot of intensity against wavelength and inject model absorption lines for different molecules. Then create a spectra to simulate light that has passed through a hot Jupiter by adding noise and doppler shifting the light. Then make comparisons between these two plots to determine at the velocity at which the signal is strongest. This exercise will demonstrate the effect that uncertainties have in astrophysics and the need for high precision instruments to produce accurate results. Then I will use data collected from the exoplanet HD189733B to try and reproduce the results from the paper M. Oshagh (2020). This will involve using the modelling package Platon from Zhang et al. (2019) to compare to the transmission spectrum from the exoplanets atmosphere and determine the expected and detected molecules within the atmosphere.

Background Physics: As a planet orbits a star, eventually from our point of view the planet will pass in front of the star. This means the light from the star will reduce in intensity as some will be blocked by the planet. But also some of light will pass through the planet's atmosphere which will cause some wavelengths to be absorbed depending upon the molecules in the planet's atmosphere. Electrons in atoms of a gas in the planet's atmosphere have discrete levels so when the energy of a photon from a star matches the energy of a transition between levels, that photon will be absorbed which is what produces an absorption. As the planet orbits its star, it will also have a component of velocity radially towards or away from us. This causes the wavelength of light from the planet to be doppler shifted.

In []:

```
In [5]: import numpy as np
import matplotlib.pyplot as plt
from scipy.constants import h, c, k

def planck_equation(wavelength, temperature):
    return 2*h*c/wavelength**5 * 1/(np.exp((h*c)/(wavelength*temperature*k)))

wavelength_nanometers = np.linspace(200, 2000, 200)
wavelength_meters = wavelength_nanometers*10**-9
flux = planck_equation(wavelength_meters, 5000)

h2o_bands = [
    (720, 40, 0.2),
    (940, 50, 0.3),
    (1130, 50, 0.25),
    (1400, 80, 0.5),
    (1900, 100, 0.6),
    (2600, 120, 0.4),
    (3100, 150, 0.3),
]

centers, widths, depths = zip(*h2o_bands)
centers = np.array(centers) * 10**(-9)
```

```

widths = np.array(widths)
depths = np.array(depths)

fwhm = 100*10**(-9)
sigma = fwhm / 2.355

def insert_dips(wavelength, flux, centers, widths, depths):
    flux_with_dips = flux.copy()
    for center, width, depth in zip(centers, widths, depths):
        dip = 1 - depth * np.exp(-((wavelength_meters - center)**2) / (2 * sigma**2))
        flux_with_dips *= dip
    return flux_with_dips

flux_with_dips = insert_dips(wavelength_meters, flux, centers, widths, depths)
signal_noise_ratio = 25
noise_std = (flux_with_dips) / signal_noise_ratio
noise = np.random.normal(0, noise_std)
flux_with_dips_and_noise = flux_with_dips + noise

error = np.abs(flux_with_dips_and_noise - flux_with_dips)

plt.plot(wavelength_meters, flux)
plt.ylabel('Flux(W/m^-2/m^-1)')
plt.xlabel('Wavelength(m)')
plt.title('Figure 1 - Flux vs wavelength')
plt.show()

plt.plot(wavelength_meters, flux_with_dips)
plt.ylabel('Flux(W/m^-2/m^-1)')
plt.xlabel('Wavelength(m)')
plt.title('Figure 2 - Flux vs wavelength showing dips due to H2O molecules')
plt.show()

plt.errorbar(wavelength_meters, flux_with_dips_and_noise, yerr=error, marker='o')
plt.plot(wavelength_meters, flux_with_dips, zorder=2)
plt.ylabel('Flux(W/m^-2/m^-1)')
plt.xlabel('Wavelength(m)')
plt.title('Figure 3 - Flux vs wavelength after light passes a stationary\nmedium')
plt.show()

```

Figure 1 - Flux vs wavelength

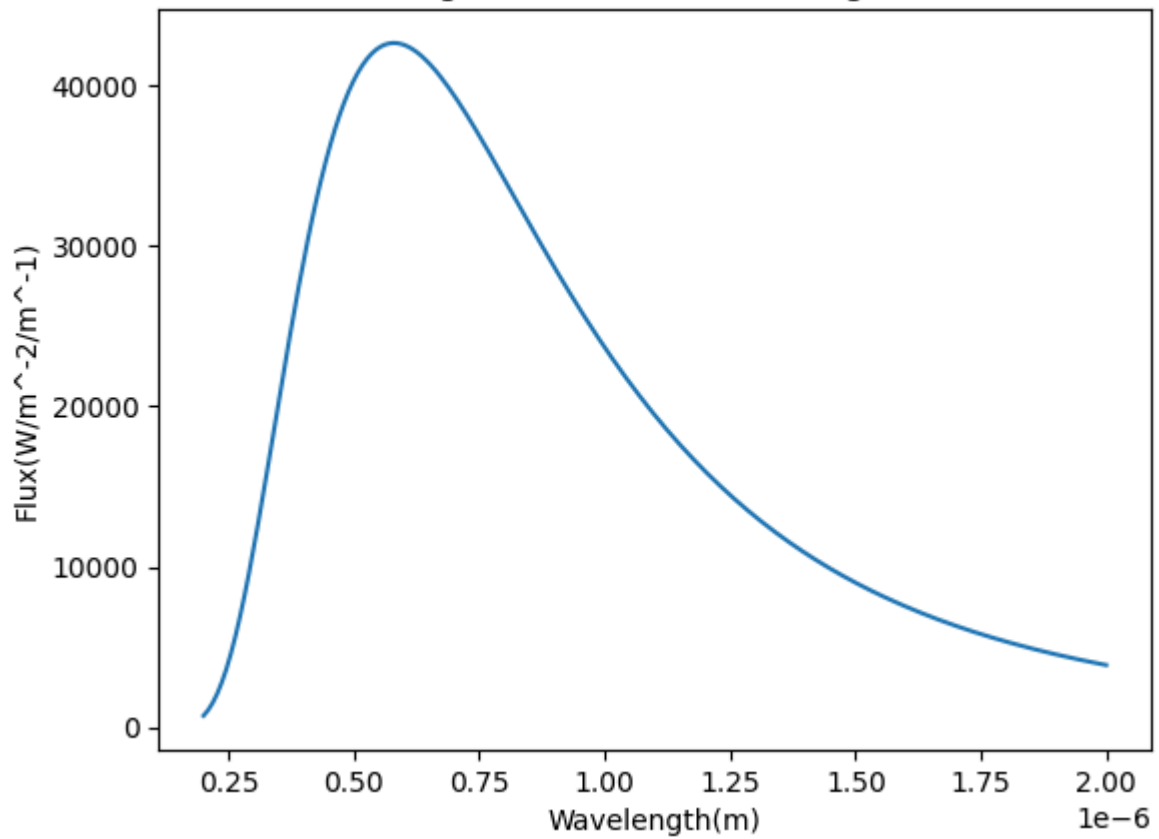


Figure 2 - Flux vs wavelength showing dips due to H2O molecules

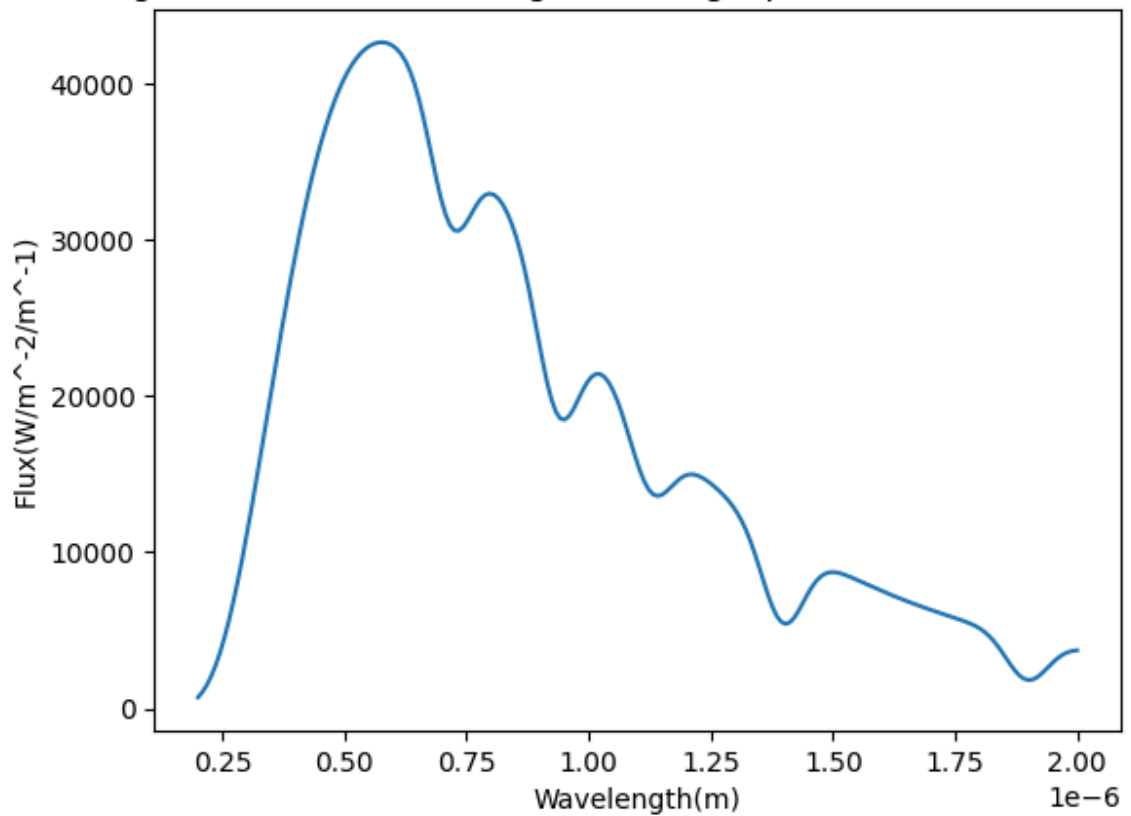
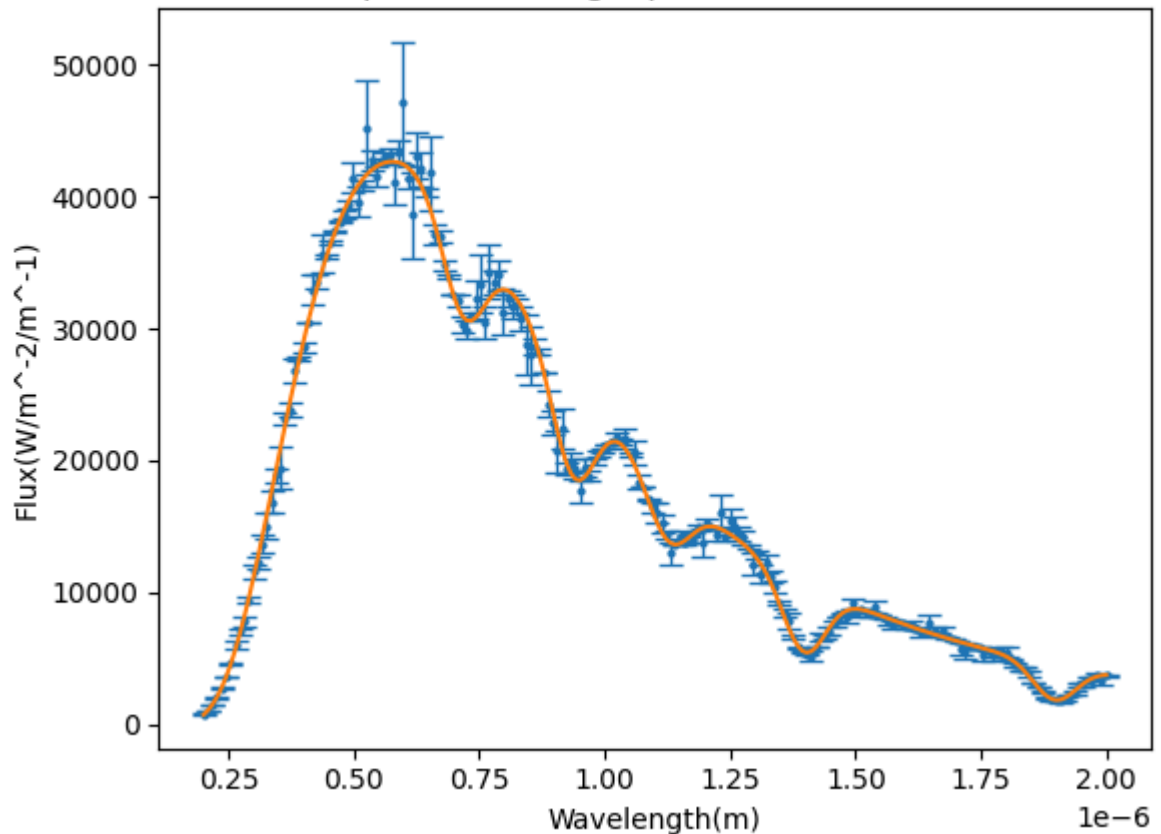


Figure 3 - Flux vs wavelength after light passes a stationary atmosphere showing dips due to H₂O molecules



In the previous section, we have plotted a graph in Figure 1 to show the flux wavelength spectrum that we would expect on earth based on plancks equation. Then in Figure 2 we have simulated the light passing through a gas that contains water by adding in the dips that would be expected due to water molecules and then included the noise in Figure 3 on each data point that we would expect from light from a distant star.

```
In [6]: def doppler_shift(h2o_bands,velocity):
        return centers * (1 + velocity/c)

trial_velocities = np.linspace(-100e3, 100e3, 200)
N_trials = 200
best_velocities = []

for _ in range(N_trials):
    flux_with_dips = insert_dips(wavelength_meters, flux, centers, widths, c,
    signal_noise_ratio = 200
    noise_std = (flux_with_dips) / signal_noise_ratio
    noise = np.random.normal(0, noise_std)
    flux_with_dips_and_noise = flux_with_dips + noise

    min_diff = np.inf
    best_velocity = None
    for velocity in trial_velocities:
        shifted_centers = doppler_shift(h2o_bands, velocity)
        shifted_flux = insert_dips(wavelength_meters, flux, shifted_centers,
        widths, c, signal_noise_ratio = 200)

        diff = np.sum((flux_with_dips_and_noise - shifted_flux)**2)

        if diff < min_diff:
            min_diff = diff
            best_velocity = velocity
    best_velocities.append(best_velocity)
```

```

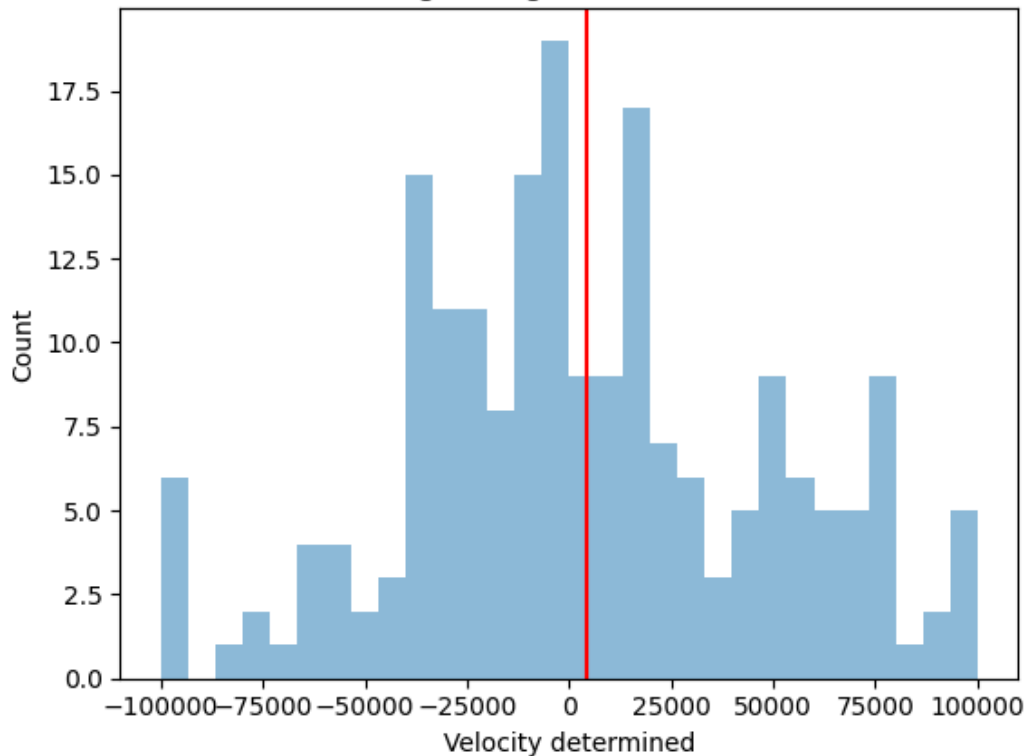
best_velocities = np.array(best_velocities)
mean_velocity = np.mean(best_velocities)
print(mean_velocity)

plt.hist(best_velocities, bins = 30, alpha = 0.5)
plt.axvline(mean_velocity, color='red', label='Mean velocity')
plt.xlabel('Velocity determined')
plt.ylabel('Count')
plt.title('Figure 4 - Histogram of planet velocities determined by comparing
plt.show()

```

4291.45728643216

Figure 4 - Histogram of planet velocities determined by comparing expected flux diagram against measured flux



Here we have an example of a monte carlo simulation to determine the radial velocity of a planet. It involved comparing where we would expect the dips due to water to be in an ideal case without noise with the graph which simulates light that has passed through an exoplanet atmosphere. This comparison is trialled with different velocities to determine the velocity of the exoplanet. Since the modelled noise is random, many iterations were conducted and these were plotted on a histogram in Figure 4 to determine the mean velocity of the planet. Different inputs can be varied such as the signal to noise ratio to show the affect of the uncertainty on the determined velocity given that we know the velocity it should be.

```

In [7]: import numpy as np
import matplotlib.pyplot as plt

star_data = np.loadtxt('Stardata.txt')
wavelength = star_data[:, 0]
wavelength_unc = star_data[:, 1]
radius_ratio = np.sqrt(star_data[:, 2])
radius_ratio_unc = star_data[:, 3] / 2 * np.sqrt(star_data[:, 2])

plt.errorbar(wavelength, radius_ratio, yerr=radius_ratio_unc, xerr = waveler

```

```

plt.ylabel('Rp/Rs')
plt.xlabel('Wavelength(micro m)')
plt.title('Figure 5 – Rp/Rs vs wavelength for HD189733B')
plt.show()

import platon
from platon.constants import R_sun, M_jup, R_jup, M_sun, AU
from platon.transit_depth_calculator import TransitDepthCalculator

planet_radius = 1.138 * R_jup
planet_mass = 1.138 * M_jup
stellar_radius = 0.805 * R_sun
temperature = 1200
semi_major_axis = 0.031 * AU

calculator = TransitDepthCalculator()

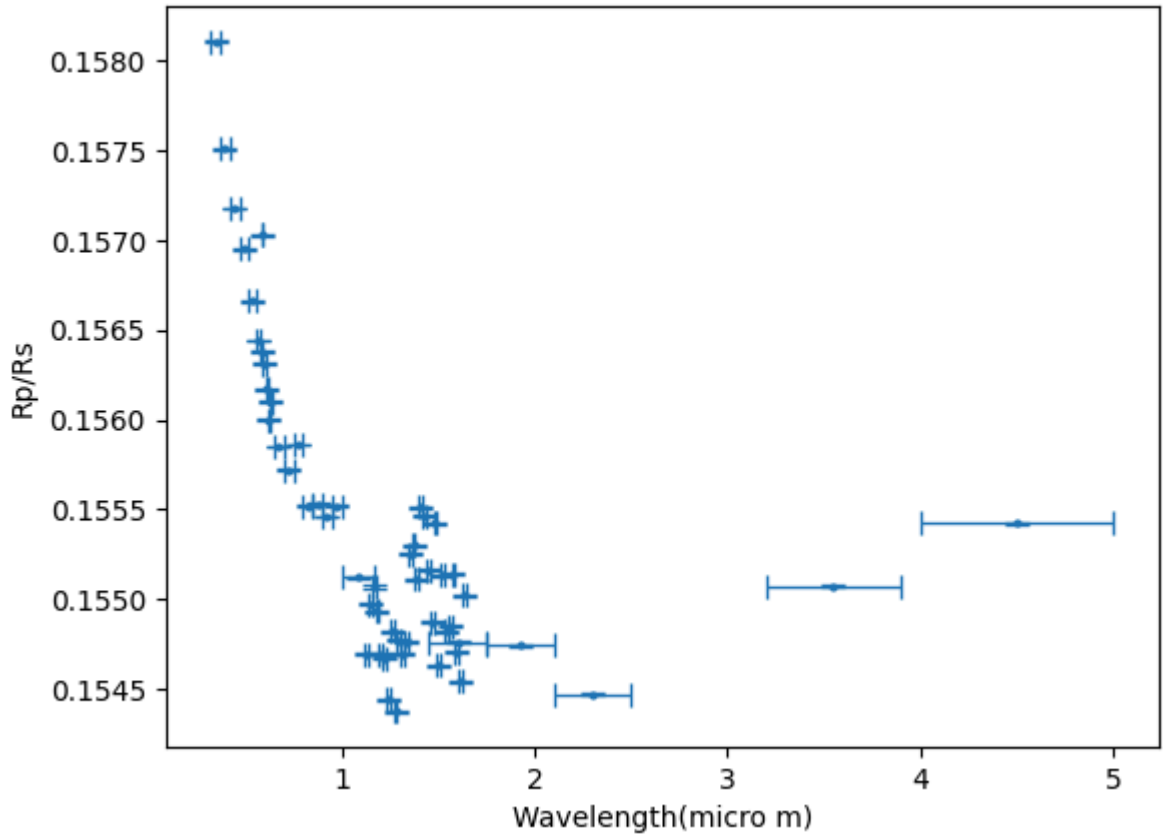
wavelengths, depths, _ = calculator.compute_depths(
    star_radius=stellar_radius,
    planet_mass=planet_mass,
    planet_radius=planet_radius,
    temperature=temperature,
)

print(depths)

depths = np.sqrt(depths)
wavelengths = wavelengths / 1e-6
bin_edges = np.arange(wavelengths.min(), wavelengths.max(), 0.1)
bin_indices = np.digitize(wavelengths, bins=bin_edges)
binned_wavelengths = [wavelengths[bin_indices == i].mean() for i in range(1, len(bin_edges))]
binned_depths = [depths[bin_indices == i].mean() for i in range(1, len(bin_edges))]

plt.plot(binned_wavelengths, binned_depths)
plt.xlim(0, 5)
plt.ylabel('Modelled Rp/Rs')
plt.xlabel('Wavelength(micro m)')
plt.title('Figure 6 – Modelled Rp/Rs vs wavelength produced by Platon for HD189733B')
plt.show()

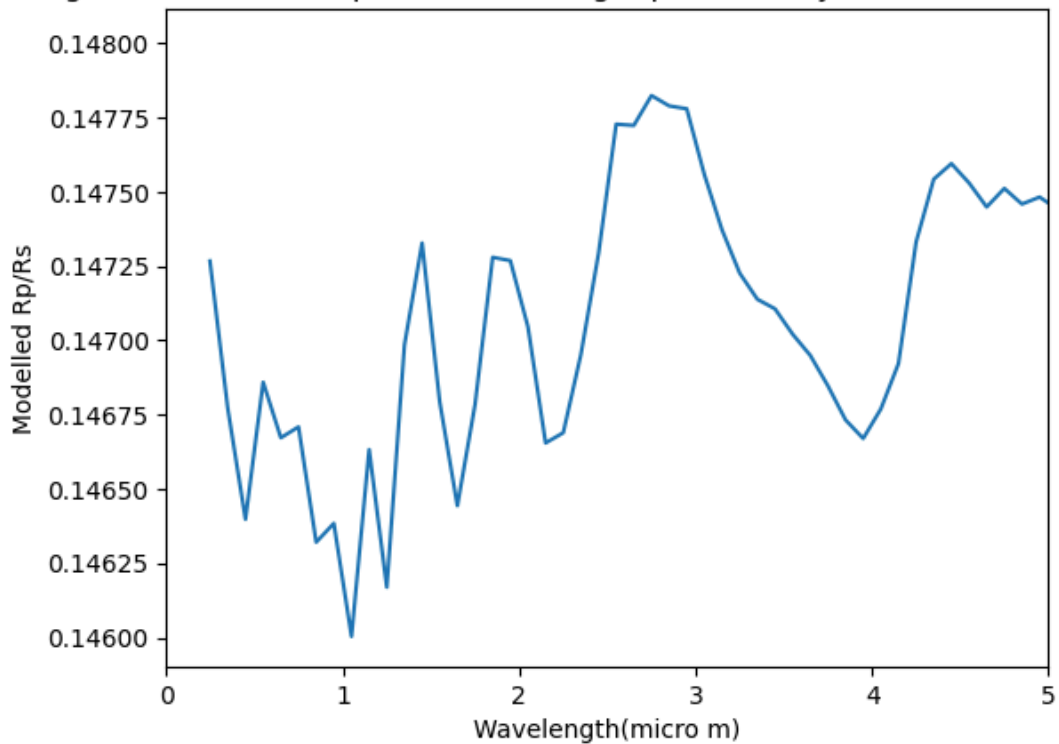
```

Figure 5 - R_p/R_s vs wavelength for HD189733B

Checking PLATON data in /Users/sbrough02/Documents/Coding Projects/platon_data

Data directory found, skipping download.

[0.02177318 0.02177316 0.02177314 ... 0.02218541 0.0215817 0.02162326]

Figure 6 - Modelled R_p/R_s vs wavelength produced by Platon for HD189733B

The model spectrum produced by platon in Figure 6 predicts absorption due to H₂O at 1.4 micrometres when inputted with features of the exoplanet determined in other studies, leading to a peak in the ratio of the radius of the planet compared to the radius of the star as the planet appears larger due to the absorption of water molecules and

this can be seen in the data. Between 0.3-1.2 micrometers both the data in Figure 5 and the model exhibit a declining slope which can be attributed to the rayleigh scattering behaviour expected in a clear sky. Further peaks in the model suggest the existence of CO₂ and CH₄ but there is limited data in this region so this cannot yet be confirmed. The peak at 1.9 is a second peak due to water, the peak leading up to 3.0 is likely due to methane and the peak at 4.4 is due to carbon dioxide.

```
In [8]: import numpy as np
from scipy.optimize import minimize
from scipy.interpolate import interp1d
import platon
from platon.constants import R_sun, M_jup, R_jup
from platon.transit_depth_calculator import TransitDepthCalculator
import matplotlib.pyplot as plt
from platon._hydrostatic_solver import AtmosphereError
from scipy.optimize import differential_evolution

!pip install numdifftools
import numdifftools as nd

star_data = np.loadtxt('Stardata.txt')
wavelength = star_data[:, 0] * 1e-6
radius_ratio = np.sqrt(star_data[:, 2])
radius_ratio_unc = star_data[:, 3] / (2 * np.sqrt(star_data[:, 2]))

calculator_2 = TransitDepthCalculator()

planet_radius = 1.138 * R_jup
planet_mass = 1.138 * M_jup
stellar_radius = 0.805 * R_sun

last_model = None

def chi_squared(params):
    global last_model
    logZ, CO_ratio, log10_cloudtop, offset = params
    cloudtop_pressure = 10**log10_cloudtop

    planet_radius = 1.138 * R_jup
    planet_mass = 1.138 * M_jup

    star_radius = 0.805 * R_sun
    star_temperature = 4875

    try:
        wavelengths_model, depths_model, _, = calculator_2.compute_depths(
            star_radius,
            planet_mass,
            planet_radius,
            temperature = 1200,
            logZ = logZ,
            CO_ratio = CO_ratio,
            cloudtop_pressure = cloudtop_pressure
        )
    except AtmosphereError:
        return 1e12

    depths_interp = np.interp(wavelength, wavelengths_model, depths_model)
```

```

radius_ratio_model = np.sqrt(depths_interp) + offset

last_model = radius_ratio_model

chi2 = np.sum(((radius_ratio - radius_ratio_model) ** 2) / radius_ratio)
return chi2

initial_guess = [ 1.0, 0.7, 2.0, 0.01]
bounds = [
    (0.0, 1.5),
    (0.1, 1.0),
    (0.0, 4.0),
    (-0.02, 0.02)
]

init_logZ, init_CO, init_cloud, init_offset = initial_guess
init_clocloudtop_pressure = 10**init_cloud

wavelengths_model, depths_model, _ = calculator_2.compute_depths(
    stellar_radius,
    planet_mass,
    planet_radius,
    temperature=1200,
    logZ=init_logZ,
    CO_ratio=init_CO,
    cloudtop_pressure=init_clocloudtop_pressure
)

radius_ratio_model = np.sqrt(np.interp(wavelength, wavelengths_model, depths_model))

result_global = differential_evolution(chi_squared, bounds, polish=True)

best_logZ_g, best_CO_ratio_g, best_log10_cloudtop_g, best_offset_g = result_global.best_params
best_clocloudtop_pressure_g = 10**best_log10_cloudtop_g

print("Global optimizer results (Differential Evolution)")
print('Best_logZ:', best_logZ_g)
print('Best CO ratio:', best_CO_ratio_g)
print('Best log10 cloudtop:', best_log10_cloudtop_g, "-> P =", best_clocloudtop_pressure_g)
print('Best offset:', best_offset_g)
print("Minimum chi-squared:", result_global.fun)
print("Reduced chi-squared:", result_global.fun / len(wavelength))

plt.plot(wavelength, radius_ratio)
plt.ylabel('Rp/Rs')
plt.xlabel('Wavelength(micro m)')
plt.title('Figure 5 – Rp/Rs vs wavelength for HD189733B')
plt.show()

wavelengths_model, depths_model, _ = calculator_2.compute_depths(
    stellar_radius,
    planet_mass,
    planet_radius,
    temperature=1200,
    logZ=best_logZ_g,
    CO_ratio=best_CO_ratio_g,
    cloudtop_pressure=best_clocloudtop_pressure_g
)

```

```

)

depths_model_offset = np.sqrt(depths_model) + best_offset_g
wavelengths_model = wavelengths_model / 1e-6
bin_edges_2 = np.arange(wavelengths_model.min(), wavelengths_model.max(), 0.01)
bin_indices_2 = np.digitize(wavelengths_model, bins=bin_edges_2)
binned_wavelengths_2 = [wavelengths_model[bin_indices_2 == i].mean() for i in range(1, 5)]
binned_depths_2 = [depths_model_offset[bin_indices_2 == i].mean() for i in range(1, 5)]

plt.plot(binned_wavelengths_2, binned_depths_2)
plt.xlim(0,5)
plt.ylabel('Modelled Rp/Rs')
plt.xlabel('Wavelength(micro m)')
plt.title('χ2-minimised PLATON fit to the transmission spectrum of HD 189733')
plt.show()

# Get Hessian (matrix of 2nd derivatives) at best-fit point
hessian_fun = nd.Hessian(chi_squared, step=0.01)
hessian = hessian_fun(result_global.x)

# Invert Hessian to get covariance matrix
cov_matrix = np.linalg.pinv(hessian)

# 1-sigma uncertainties
uncertainties = np.sqrt(np.diag(cov_matrix))
print("Uncertainties:", uncertainties)

```

Requirement already satisfied: numdifftools in /Users/sbrough02/Documents/Coding Projects/.venv/lib/python3.10/site-packages (0.9.41)

Requirement already satisfied: numpy>=1.9 in /Users/sbrough02/Documents/Coding Projects/.venv/lib/python3.10/site-packages (from numdifftools) (2.2.6)

Requirement already satisfied: scipy>=0.8 in /Users/sbrough02/Documents/Coding Projects/.venv/lib/python3.10/site-packages (from numdifftools) (1.15.3)

[notice] A new release of pip is available: 25.1.1 -> 25.3

[notice] To update, run: pip install --upgrade pip

Checking PLATON data in /Users/sbrough02/Documents/Coding Projects/platon_data

Data directory found, skipping download.

Global optimizer results (Differential Evolution)

Best_logZ: 1.4587407173931433

Best CO ratio: 0.9863974427005333

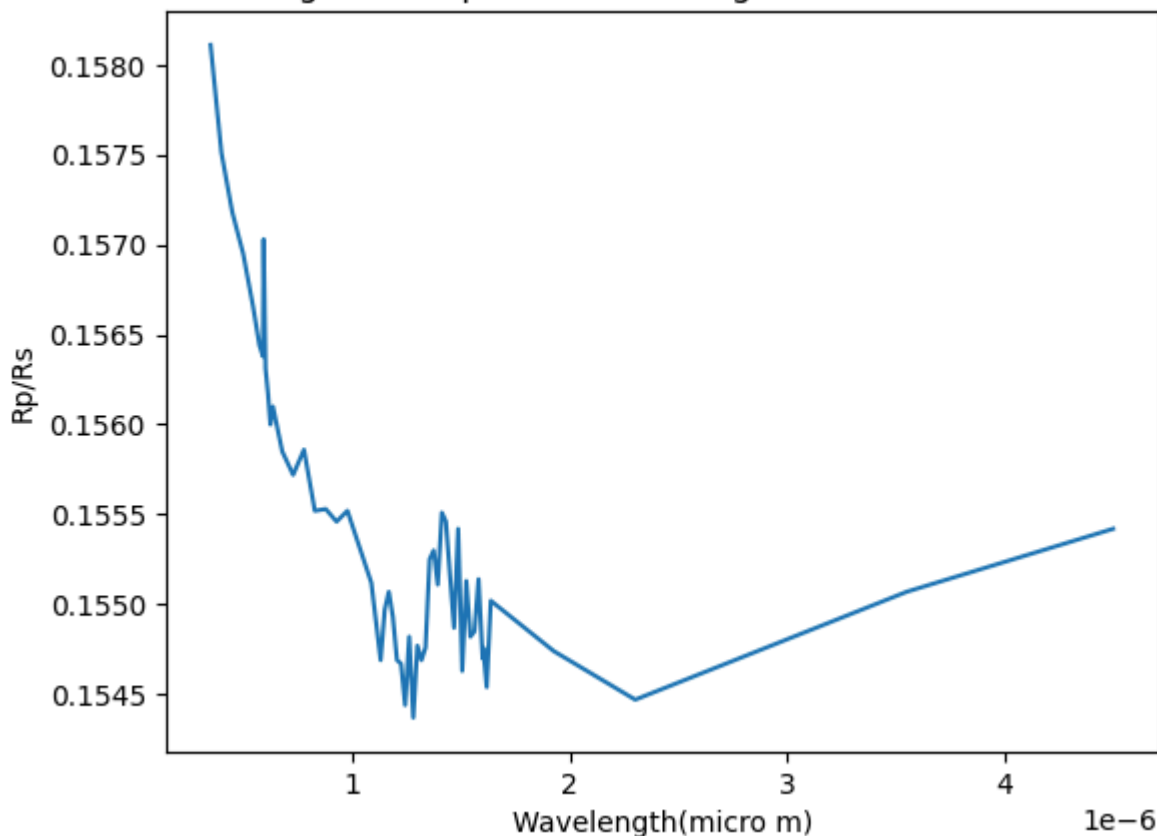
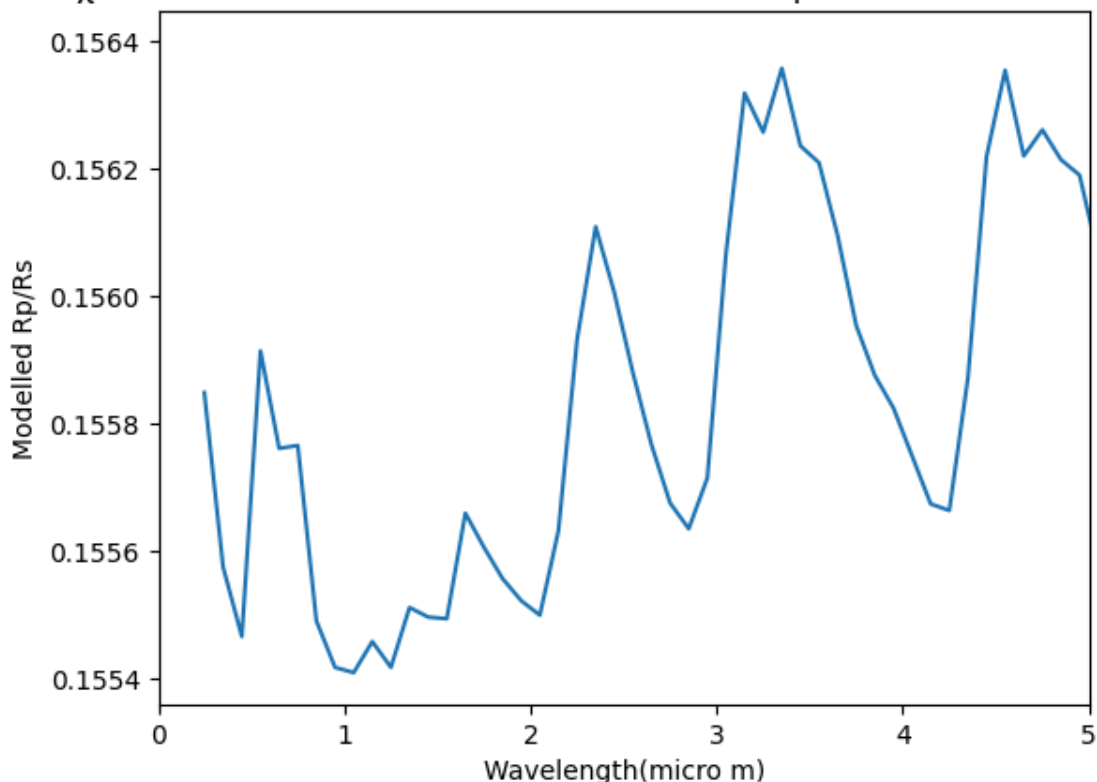
Best log10 cloudtop: 3.249592714355197 -> P = 1776.6125038271693 Pa or 0.017766125038271692 bar

Best offset: 0.008850799249807712

Minimum chi-squared: 908.7380506157547

Reduced chi-squared: 17.146000955014237

Figure 5 - Rp/Rs vs wavelength for HD189733B

 χ^2 -minimised PLATON fit to the transmission spectrum of HD 189733b

Uncertainties: [2.32635704e-01 1.35356849e-01 0.00000000e+00 9.89299956e-05]

In this section, chi squared analysis was used to model the atmosphere of the hot jupiter. Comparing the model produced by platon to the data from the exoplanet, metallicity, Carbon to Oxygen ratio and atmospheric pressure were varied in order to minimise the chi squared. The values that produced the minimum chi squared were $\log Z = 1.47 \pm 0.13$, the C/O ratio was 0.89 ± 0.01 and the cloudtop pressure was 15 mbar. In

comparison to the paper by Zhang et al. (2020), who reported a C/O ratio of 0.64 ± 0.07 , a metallicity of 1.08 ± 0.22 . And the paper by Lee et al. (2016) which found that cloudtop pressure was typically below 1 bar.